EKVC SEASON 4 • INNOVATION ROUND

Kurukshetra Kart Master Build Document

IC Go-Kart Telemetry, Digital Twin & Predictive Systems

PREPARED BY	**Kurukshetra Racing, SKCET**
COMPETITION	EKVC Season 4 — Innovation Round
TEAM	Embedded Systems • Telemetry • Data Analytics • Dynamics & Testing
VERSION	1.0 — Master Build Manual
TIMELINE	Semester-based (3–6 months, phase-gated)
LAST UPDATED	September 2026

> "Real kart. Real data. Real model. Real validation. The closed loop."
>
> Measure → model → predict → validate → improve.

Contents

00	Executive Summary	Scope, commitments, cost
01	System Architecture Overview	Five-layer stack, data flow
02	Hardware Master Plan	BOM, signals, risks
03	Software & Tech Stack	Firmware, dashboard, pipeline
04	Predictive Collision Awareness	TTC algorithm, warning zones
05	FastF1 Integration & Reference Harness	Reference data, not input
06	APIs & Data Interfaces	Packets, SD format, schema
07	Digital Twin & Calibration Loop	Four models, validation
08	Bottlenecks & Critical R&D Zones	Track position, noise, loss
09	R&D Zones — Candidates & Stretch Goals	Options with trade-offs
10	Achievable vs. Not Achievable	Honest scope boundaries
11	Semester Roadmap & Phase Gates	Six phases, go/no-go
12	Team Roles & Ownership	Seven subteams
13	Cost Breakdown & Budget	V1 BOM, V2/V3 expansion
14	Realistic vs. Aspirational Outcomes	Deliver / might / won't
15	Technical Documentation Standards	Repo layout, review rules
16	Final Recommendations & Success Criteria	Demo sequence, pitch
A	Appendix A — ESP32 Pin Mapping	Quick reference
B	Appendix B — FastF1 Throttle Extraction	Worked example
C	Appendix C — Validation Checklist	Weekly use
D	Appendix D — Dead Reckoning Algorithm	V1 track position

00 Executive Summary

> Kurukshetra Kart is a complete telemetry-to-prediction platform built around a low-cost embedded stack (ESP32 only) that turns real go-kart performance data into a validated physics-based digital twin.

The system closes an engineering feedback loop: measure real kart behaviour → build a model → test virtual scenarios → validate predictions → improve the real setup.

CORE INNOVATION

The closed-loop integration of predictive collision awareness (time-to-contact driven) with a calibrated physics simulator, supported by a disciplined data pipeline and honest validation methodology. This is not a sensor-by-sensor gadget — it is an end-to-end engineering tool.

Architecture in One Line

ESP32 collects nine core telemetry streams → logs locally to SD card → sends a live subset wirelessly to the pit → Python ingests and models → the simulator tests alternate conditions → results feed back into setup optimisation.

TABLE 0.1 — KEY TECHNICAL COMMITMENTS

AREA	COMMITMENT
Controller	ESP32 (dual-core, 240 MHz) — Arduino Nano dropped entirely
Data strategy	Full local logging to SD card; compact wireless subset at 10–20 Hz
Simulation	Physics-based point-mass model, validated against real telemetry
Track position	Flagged as an open R&D problem; three candidate solutions proposed (RTK-GPS, dead reckoning, sensor fusion) — see Section 8
Cost	Realistic BOM ₹18,000–24,000 (core hardware + wireless + data + integration)
Delivery	Layered: V1 core telemetry + display → V2 wireless + dashboard → V3 digital twin + advanced ML
Team	Seven functional subteams with clear ownership; no overlapping authority on subsystems

01 System Architecture Overview

1.1 The Five-Layer Stack

Data flows upward through five layers. Each layer has a distinct owner team and can operate independently.

TABLE 1.1 — LAYER RESPONSIBILITIES

#	LAYER	FUNCTION	OWNER
5	**Optimization & Engineering**	Setup analysis, what-if testing; recommends chassis / wing / brake changes	Dynamics + Testing
4	**Digital Twin & Simulation**	Python physics engine: track model, kart model, driver model	Vehicle Dynamics
3	**Data & Analytics Pipeline**	Cleaning, sync, track reconstruction, lap analysis, sector comparison	Python / Data
2	**Telemetry & Live Display**	ESP32 firmware, wireless, driver HUD, SD logging, real-time alerts	Embedded + Telemetry
1	**Vehicle Sensors & Data Sources**	9 sensor channels, ESP32 acquisition: wheel speed, RPM, temp, steering	Embedded + Mechanical

> CRITICAL DESIGN PRINCIPLE
>
> The kart continues to log to SD even if wireless fails. **Wireless telemetry is useful; SD logging is mandatory.**

1.2 Data Flow Summary

TABLE 1.2 — STAGE-BY-STAGE FLOW

STAGE	WHAT HAPPENS
On-kart	Sensors → ESP32 → simultaneous SD logging + wireless packet creation
To pit	Compact live packet (speed, RPM, temperature, steering, G-force, TTC, warning flag) at 10–20 Hz
Post-run	Full SD dataset (all channels, 50–100 Hz) imported into Python
Analysis	Cleaned dataset → track reconstruction → lap/sector alignment → virtual modelling
Simulation	Model tested against real data; predictions compared to measured corner speeds
Feedback	Validated setup recommendations return to the real kart for the next run

02 Hardware Master Plan

2.1 Core Components — V1 (Semester Target)

TABLE 2.1 — V1 COMPONENT REGISTER

SYSTEM	COMPONENT	PURPOSE	QTY	COST (₹)	NOTES
Controller	ESP32 Dev Board	Central acquisition, filtering, comms	1	400–600	Dual-core, 240 MHz; drop Arduino Nano
Wheel Speed	Hall-effect sensor + magnet	Vehicle speed, wheel slip	1–2	80–150 ea	Proven on multiple karts
Engine RPM	Inductive pickup / tach sensor	Engine operating point	1	150–300	Signal conditioning required
Engine Temp	DS18B20 probe	Thermal monitoring	1	80–120	±0.5 °C precision
Steering Angle	Rotary potentiometer / encoder	Driver input, corner analysis	1	50–100	Linear pot on steering column
G-Force / IMU	MPU6050 or ICM-42688	Accel, gyro, yaw rate	1	120–200	Essential for dynamics validation
Obstruction / TTC	HC-SR04 (V1) or ToF (V2)	Closing speed, collision warning	1	80–150	Ultrasonic for proof-of-concept
Throttle Position	Hall throttle sensor or pot	Driver input for modelling	1	100–200	Intake manifold mount
Brake Pressure	Brake line transducer	Braking dynamics V2+	1	400–800	V2 addition; high reliability needed
Data Logging	MicroSD module + card	Full dataset archival	1	150–250	32 GB card, sustained 50 Hz logging
Display (driver)	0.96" OLED (I2C)	Steering-wheel HUD	1	180–250	Minimal; speed / RPM / alert only
Wireless Link	nRF24L01+ or ESP-NOW	Live telemetry to pit	1	150–300	10–20 Hz packet, 100 m range
Power Regulation	7805 + caps / isolated buck	Clean 5 V for ESP32 + sensors	1	200–350	Signal isolation from engine noise critical
Mounting / Integration	3D printed enclosures, shielded cables	Vibration & EMI protection	—	500–1000	Sensor mounts, cable routing, ground plane
Connectors / Wiring	M5/M6 plugs, shielded twisted pair	Reliable sensor interfaces	—	300–500	Waterproof connectors, grounding scheme
V1 SUBTOTAL — core telemetry stack				3,400–5,500	

2.2 Staged Expansion — V2 and V3

TABLE 2.2 — EXPANSION ROADMAP

STAGE	ADDITIONS
V2 **Weeks 4–5 in**	Brake pressure sensor (₹400–800) • GPS/GNSS module (standard GPS is insufficient on tight kart tracks — see Section 8) • secondary wheel sensor for full 4-wheel speed • higher-resolution IMU (ICM-42688 or similar) • improved distance sensor (VL53L0X ToF)
V3 **Week 8+ — digital twin grade**	Suspension travel sensors (linear pots) • tire temperature sensors (IR, per wheel) • tire pressure sensors (TPMS-grade) • multi-camera setup (GoPro + onboard stereo) • advanced GNSS with RTK capability

2.3 The 10 Signals That Matter Most

If the system had to function with only ten channels, these are the ones.

TABLE 2.3 — PRIORITY SIGNAL LIST

#	SIGNAL	WHY IT MATTERS
1	**Wheel Speed**	Vehicle reference; fundamental to lap time and slip
2	**Engine RPM**	Power availability, gear point
3	**Steering Angle**	Corner characterisation

4	**Longitudinal Acceleration**	Braking and traction
5	**Lateral Acceleration**	Cornering speed and limit
6	**Yaw Rate**	Rotation dynamics
7	**Throttle Position**	Driver input for model training
8	**Engine Temperature**	Thermal state
9	**Time-to-Contact (TTC)**	Obstruction-based driver warning
10	**Track Position**	Lap segmentation — **open R&D problem**

2.4 Signal Acquisition & Timestamping Strategy

> **CRITICAL RULE**
>
> Every measurement carries a timestamp from the **same reference clock**.

- ESP32 internal timer: microsecond resolution
- Sensors sampled at 50–100 Hz (10–20 ms interval)
- Data packed with timestamp before SD write and wireless transmission
- Post-processing: all channels aligned to the same timeline

Why this matters: comparing wheel speed at time *T* against steering angle at the same *T* requires synchronised clocks, not guessing. Cross-channel desynchronisation introduces false correlations in the model.

2.5 Hardware Risks & Mitigations

TABLE 2.4 — RISK REGISTER

RISK	IMPACT	MITIGATION
Engine electrical noise → corrupted sensor signals	Kart stops logging mid-run	Shielded wiring, isolated 5 V, high-quality caps on ESP32 supply
Vibration → solder joint failures	Intermittent sensor dropout	Vibration-rated mounts, potted connectors, strain relief
Data loss due to SD write failure	Entire run data gone	Checksums on every record; watchdog timer resets ESP32 if stuck
Wireless dropout mid-run	Pit loses live telemetry (kart keeps logging)	Graceful degradation: pit shows last-known state; full data arrives post-run
Throttle / brake sensor calibration drift	Setup recommendations based on wrong input	Bench-check sensors before every track session

03 Software & Tech Stack

3.1 Embedded Firmware (ESP32)

Language: C/C++ (Arduino framework or ESP-IDF)

TABLE 3.1 — FIRMWARE RESPONSIBILITIES

RESPONSIBILITY	DETAIL
Sensor acquisition	I2C, analog and digital reads across all nine channels
Real-time computation	Filtering and derived values — speed, acceleration, TTC
Timestamp synchronisation	Single microsecond reference clock for every record
SD card logging	Binary (fast) or CSV (inspectable) format
Wireless transmission	Compact packet creation and send
Driver display	OLED HUD updates
Fault detection	Sensor dropout, low battery, temperature threshold

TASK ARCHITECTURE

```
FreeRTOS (dual-core scheduling)
├─ Core 0: Sensor acquisition (interrupt-driven, ~5 ms cycle)
├─ Core 1: Data filtering, derived values, display + wireless
└─ Background: SD write with ring buffer (no blocking I/O)
```

TABLE 3.2 — KEY ROUTINES

ROUTINE	FUNCTION
`acquire_sensors()`	Poll all 9 channels, timestamp each
`compute_derived()`	Speed, acceleration, TTC calculations
`filter_data()`	Moving average, outlier rejection
`sd_log()`	Binary write to SD (pre-allocated, fast)
`wireless_send()`	Compact packet formation
`oled_update()`	Minimal driver display

> **LATENCY TARGET**
>
> Sensor read → calculation → SD write **≤ 20 ms**. Wireless **≤ 50 ms**.

3.2 Pit Dashboard (Python / Web)

Backend: Python + Flask / FastAPI. **Purpose:** receive wireless packets, display live gauges, log session metadata.

TABLE 3.3 — DASHBOARD COMPONENTS & STACK

ELEMENT	CHOICE / DETAIL
Wireless receiver	ESP32 or laptop USB adapter
Live gauges	Speed, RPM, temperature, TTC
Alerts	High temp, TTC warning, sensor fault
Session recording	Start/stop, metadata capture

Visualization	Streamlit (rapid prototype) or React (production)
Database	SQLite — sessions, run metadata
Graphing	Plotly for live traces

3.3 Data & Analytics Pipeline (Python)

Input: SD card data (CSV or binary). **Output:** cleaned, lap-aligned, model-ready dataset with sector comparisons.

TABLE 3.4 — PROCESSING STAGES

#	STAGE	OPERATION
1	**Ingest**	Read raw SD file, validate timestamps
2	**Clean**	Remove dropouts, interpolate glitches, filter outliers
3	**Synchronise**	Align all channels to master timestamp
4	**Track Reconstruction**	Infer lap position from GPS or dead reckoning (Section 8)
5	**Lap Segmentation**	Identify start/finish line, break into laps
6	**Sector Analysis**	Compare lap sectors, identify performance deltas

Libraries: pandas, numpy, scipy, matplotlib / plotly.

3.4 Digital Twin & Physics Model (Python)

Scope: V1 is a point-mass simulator; V2 can add suspension dynamics.

TABLE 3.5 — MODEL INPUTS AND OUTPUTS

INPUTS	OUTPUTS
Track geometry (centreline, curvature, elevation per section)	Predicted lap time
Kart parameters (mass, wheelbase, max grip, drag coefficient, torque curve)	Speed through each corner
Driver behaviour (throttle/brake pattern per corner, steering rate)	G-force envelope
Environment (temperature, humidity, surface grip)	Throttle and brake zones

TABLE 3.6 — CALIBRATION LOOP

STEP	ACTION
1	Measure a real lap; extract throttle and braking pattern
2	Simulate the lap with a guessed tire grip value
3	Compare simulated corner speed against actual
4	Adjust the grip parameter until the match is within ±5%
5	Use the calibrated model to test “what-if” scenarios

Physics engine: custom point-mass model (F = ma, friction circle limits, drag equation).

3.5 Recommended Technology Choices

TABLE 3.7 — STACK DECISIONS

LAYER	TECHNOLOGY	WHY

Embedded	C++ / ESP-IDF	Real-time, proven on this hardware
Pit Dashboard	Python + Streamlit	Rapid iteration, good enough for a student project
Analytics	pandas + numpy	Standard, fast, well documented
Simulation	scipy + matplotlib	No need for commercial tools (Chrono, Assetto Corsa)
Version Control	Git + GitHub	Mandatory for team coordination

3.6 Data Format & Exchange

On-kart (SD card): binary format for speed, CSV option for human inspection.

```
{
  "timestamp_us":     12450000,
  "wheel_speed_kmh":  58.3,
  "rpm":              7240,
  "temp_c":           81.5,
  "steering_deg":     13.4,
  "throttle_pct":     82,
  "accel_x_g":        0.72,
  "accel_y_g":        0.91,
  "yaw_rate_dps":     18.4,
  "distance_cm":      142,
  "ttc_s":            2.14,
  "warning_flag":     0
}
```

Wireless (pit): compact binary, ≈20 bytes per packet at 20 Hz ≈ 400 bytes/sec.

04 Predictive Collision Awareness System

4.1 The Innovation Round Feature

> This subsystem calculates time-to-contact (TTC) in real time and alerts the driver through a steering-wheel-mounted display. It is **not** an autonomous braking system — it is driver assistance.

4.2 Core Algorithm

```
closing_speed   = (previous_distance - current_distance) / time_elapsed
time_to_contact = current_distance / closing_speed        (if closing_speed > 0)
```

TABLE 4.1 — WARNING ZONES (TUNED DURING TRACK TESTING)

ZONE	CONDITION	DRIVER OUTPUT
SAFE	TTC > 2.5 s, or object moving away	Green display; no alert
CAUTION	TTC 1.0–2.5 s	Yellow steady display + visual indicator
DANGER	TTC < 1.0 s	Red flashing + buzzer

4.3 Hardware Implementation

TABLE 4.2 — SUBSYSTEM HARDWARE

ELEMENT	SPECIFICATION
Sensor	HC-SR04 ultrasonic V1 or VL53L0X ToF V2
Display	0.96" OLED on steering wheel hub
Alert	Buzzer + LED for redundancy
Response latency	< 50 ms (sense → compute → display)

4.4 Honest Limitations & Testing

TABLE 4.3 — CAPABILITY BOUNDARY

CLAIM	STATEMENT
DOES NOT	Autonomously control the kart; distinguish a real threat from a false positive (e.g. a trackside cone)
DOES	Calculate how fast the gap is closing and warn proportionally
Track testing	Controlled obstacle approach; measure actual latency; report measured numbers to judges, not estimates
Stretch goal	Steering angle cross-check to reduce false alarms when the driver is clearly steering away

05 FastF1 Integration & Data Reference Harness

5.1 What FastF1 Actually Is

FastF1 is a Python library that fetches official Formula 1 telemetry from FIA public feeds and season archives. **It is not something we feed into; it is something we learn from.**

5.2 How We Use It

TABLE 5.1 — FOUR USES OF FASTF1

USE	DETAIL
Reference structure	FastF1 data has exactly the time-aligned structure we need (speed, RPM, throttle, braking, steering) but at higher quality — professional sensors at 1000 Hz
Prototyping	Download a real F1 lap, extract throttle/braking patterns, use them to train our simulator
Validation benchmark	Compare our kart model against F1-style lap analysis — sector-by-sector, not absolute speed
Pipeline development	The code we write for FastF1 data teaches us how to structure our own pipeline

5.3 Example Workflow

```
import fastf1
import pandas as pd

# Download a real lap (example: Monaco GP 2023)
session = fastf1.get_session(2023, 'Monaco', 'R')
lap = session.laps.pick_driver('VER').get_car_data().pick_stint(1)

# Extract throttle/brake pattern
throttle_pattern = lap[['Time', 'Throttle', 'Brake', 'Speed']].copy()

# Seed our kart simulator:
#   F1:   "At this corner, driver applies 82% throttle while doing 150 km/h"
#   Kart: "What speed do we do at this corner if we apply 82% throttle?"
#   Then: compare predicted vs. actual -> calibrate grip
```

CLARITY — WE DO NOT FEED OUR DATA INTO FASTF1

Our kart telemetry is a separate dataset. FastF1 is a teaching tool and a reference, not an input source. Our pipeline is **bidirectional** (measure → model → predict → validate on the real kart); FastF1 is **read-only** reference data.

06 APIs & Data Interfaces

6.1 ESP32 ↔ Pit Wireless Protocol

TABLE 6.1 — LIVE PACKET STRUCTURE (16 BYTES TOTAL)

FIELD	BYTES	CONTENTS
Timestamp	4	Microsecond reference clock
Speed	2	Wheel speed, km/h
RPM	2	Engine operating point
Temperature	1	Engine temp, °C
Steering	1	Steering angle, degrees
Throttle	1	Throttle position, %
Accel X/Y	2	Longitudinal + lateral G
TTC	2	Time-to-contact, seconds
Flags	1	Warning / fault bits
TOTAL	16	**10–20 Hz • 50–100 m LOS (nRF24L01+)**

Redundancy: if wireless drops, the kart continues logging and the pit displays the last-known state.

6.2 SD Card Format

TABLE 6.2 — FORMAT OPTIONS

OPTION	STRUCTURE	TRADE-OFF
A — Binary (fast)	`[ts 4B][speed 2B][rpm 2B][temp 1B] ... [checksum 1B]`	Pre-allocated 1 MB chunks for speed; not human-readable
B — CSV (readable)	`timestamp_us,speed_kmh,rpm,temp_c,steering_deg,...`	Inspectable in any editor; slower writes, larger files
RECOMMENDATION — **log in binary for speed, convert to CSV post-run for analysis**		

6.3 Python Import Interface

```
import kurukshetra_kart as kk

# Load a session
session = kk.Session.from_sd_card("session_001.bin")

# Access telemetry
speed = session.telemetry['speed']
rpm   = session.telemetry['rpm']

# Align to laps
laps = session.get_laps()
lap1 = laps[0]
lap1.plot_speed()
```

6.4 Database Schema (SQLite)

```
-- Sessions: metadata for each track outing
CREATE TABLE sessions (
    id              INTEGER PRIMARY KEY,
    date            DATETIME,
    track_name      TEXT,
    driver_name     TEXT,
    num_laps        INTEGER,
    avg_lap_time    REAL,
    telemetry_file  TEXT
);

-- Laps: individual lap records
CREATE TABLE laps (
    id           INTEGER PRIMARY KEY,
    session_id   INTEGER,
    lap_number   INTEGER,
    lap_time     REAL,
    sector_times TEXT,          -- JSON array
    FOREIGN KEY (session_id) REFERENCES sessions(id)
);
```

07 Digital Twin & Calibration Loop

7.1 The Four Models

TABLE 7.1 — DIGITAL TWIN SUB-MODELS

MODEL	REPRESENTS	COMPLEXITY	DATA SOURCE
Track	Centreline, curvature, elevation, surface grip	Geometry from GPS traces or manual mapping	Real laps (wheel speed + steering)
Kart	Mass, wheelbase, max lateral G, engine power curve, drag	Spec sheet + calibration	Measured acceleration vs. throttle
Environment	Temperature, humidity, wind, track surface condition	Weather API + manual input	BME280 sensor + pit notes
Driver	Throttle/brake timing, steering rate, corner speeds	Behaviour extracted from telemetry	Raw throttle/brake logs

7.2 V1 Simulator — Point-Mass Model

Scope: not a full vehicle dynamics engine — a simple, buildable model that validates against real telemetry.

TABLE 7.2 — GOVERNING EQUATIONS

QUANTITY	EXPRESSION	NOTE
Lateral grip	`G_max = μ × 9.81 m/s²`	Tire-dependent, tuned via calibration
Corner speed	`V = √(R × G_max × 9.81)`	R = corner radius
Drag	`F_drag = 0.5 × ρ × A × Cd × V²`	Cd tuned during calibration
Acceleration	`a = (P_engine / V - F_drag) / mass`	Straight-line power limit

SIMULATION LOOP

```
1. Load track (segments: distance, radius, elevation)
2. For each segment:
       if straight -> accelerate at max power
       if corner   -> limit speed to grip circle
       integrate distance and time
3. Output: predicted lap time, speed trace, G-force envelope
```

7.3 Calibration Against Real Data

Goal: adjust the unknown parameters (tire grip μ, drag Cd) so predicted corner speeds match measured corner speeds.

```
1. Extract one real lap from SD card
2. Identify corners (high steering angle + lateral accel)
3. For each corner:
       measured  = corner speed (wheel speed + GPS/dead reckoning)
       predicted = speed from grip model
       error     = measured - predicted
4. Adjust grip estimate:
       mu_new = mu_old * (measured_speed / predicted_speed)
5. Re-simulate, compare error
6. Repeat until error < 5%  (or accept as "good enough")
```

7.4 Validation Methodology

TABLE 7.3 — CLAIMS DISCIPLINE

POSITION	STATEMENT
WE WILL NOT CLAIM	Zero latency or instantaneous response • guaranteed lap-time improvement • perfect prediction in all conditions
WE WILL PROVE	Model error < ±5% on corners, validated against measured telemetry • consistent predictions across multiple runs on the same track • reproducibility (same conditions → same predicted result)

TEST MATRIX

- 2–3 real laps with measured data
- Calibrate the model on lap 1
- Predict lap 2 and lap 3 speeds
- Report measured vs. predicted for each corner
- Calculate RMSE (root-mean-square error)

7.5 What-If Simulation (Scenario Testing)

TABLE 7.4 — SCENARIOS THE CALIBRATED MODEL CAN ANSWER

SCENARIO	QUESTION	OUTPUT
Speed sweep	What if we did this corner at 55 km/h instead of 50?	Required G-force, risk of spin
Setup change	Does softer suspension give better grip?	Predicted lap time delta
Surface condition	What if the track is wet instead of dry?	Grip reduction factor, revised lap time
Driver input	What if we brake 50 m earlier?	Entry speed change, apex speed change

> **IMPORTANT**
>
> These are **predictions, not guarantees**. Always validate on track before trusting a simulator output.

08 Bottlenecks & Critical R&D Zones

8.1 Track Position Accuracy — OPEN PROBLEM

The challenge: know, at any moment, where on the track the kart is.

Why it matters: lap segmentation (identifying start/finish and lap boundaries), sector analysis (which section was slow?), and digital twin validation (comparing model predictions to specific track regions).

TABLE 8.1 — LOCALISATION METHODS COMPARED

METHOD	ACCURACY	COST (₹)	LIMITATION
Standard GPS	±5–10 m	1,500–2,000	Far too coarse for a tight kart track
Dead reckoning (steering + wheel speed integration)	±2–5 m (drifts over time)	200	Cumulative error across a 1 km lap
RTK-GPS (real-time kinematic)	±2–5 cm	8,000–15,000	Very accurate, but expensive for prototyping
LiDAR + SLAM	±0.5 m	20,000+	Over-engineered for a student project
Sensor fusion (Kalman: GPS + IMU + wheel odometry)	±1–2 m	2,500–4,000	Realistic; requires calibration

8.2 Proposed Solution Path (Semester 1)

TABLE 8.2 — PHASED LOCALISATION PLAN

PHASE	TIMING	APPROACH	OUTCOME
Phase 1	Week 2–3	Dead reckoning	Integrate steering angle and wheel speed; accept ±2–5 m error — sufficient for rough sector analysis
Phase 2	Week 5–6	Sensor fusion	Combine wheel speed + steering + standard (or phone) GPS through a Kalman filter; drift reduced to ±1–2 m
Phase 3	V2, if time allows	RTK-GPS or manual mapping	Hire RTK equipment for 1–2 sessions for a precise baseline; or walk the track recording phone-GPS waypoints to build a reference centreline

> DO NOT ATTEMPT IN V1
>
> Full LiDAR SLAM or custom vision-based localisation. Out of scope.

8.3 Engine Electrical Noise & Signal Integrity

The challenge: the ignition system (12 V, high-frequency switching) radiates noise that corrupts sensor signals.

Symptoms: temperature sensor reads wildly (80 °C, then 150 °C, then 80 °C); RPM count jumps erratically; wireless dropouts during acceleration.

TABLE 8.3 — MITIGATIONS IN ORDER OF IMPORTANCE

RANK	SOLUTION	IMPLEMENTATION
1	**Shielded cables**	Twisted pair, ferrite cores on sensor leads
2	**Isolated power supply**	Separate clean 5 V for microcontroller and sensors, not shared with ignition
3	**Capacitor bank**	100 µF + 10 µF close to ESP32 power pins
4	**Grounding scheme**	Single-point ground at battery negative; no multiple ground loops
5	**RF shielding**	Aluminium enclosure for ESP32, not an open breadboard

Testing: power up the kart engine and monitor sensor readings. Acceptable if variation is < 2% during idle.

8.4 Wireless Packet Loss & Redundancy

TABLE 8.4 — FAILURE BEHAVIOUR

OUTCOME	BEHAVIOUR
ACCEPTABLE	Pit displays “signal lost” and shows last-known state. Kart continues logging to SD. Full data arrives after the run.
NOT ACCEPTABLE	Pit believes the kart is stopped when it is still running; or the driver HUD goes blank.

MITIGATIONS

- Watchdog timer on ESP32 — auto-reset if firmware hangs
- Redundant alert channel — buzzer on kart, independent of wireless
- Pre-allocate the wireless buffer; allow local operation if transmit fails
- Test range in an open field before the first track session (ensure 100 m coverage)

8.5 Model Validation Without Crashing

The challenge: the simulator may predict behaviour that does not happen — e.g. “this corner is flat at 80 km/h” when the real kart can only manage 65.

Solution: always test simulator recommendations on the kart before trusting them. Start conservative (slower speeds, softer inputs), measure, then refine.

```
1. Simulate : "Try 2 degrees less brake bias"
2. Measure  : "What does the kart actually do with that bias?"
3. Compare
4. Next iteration
```

8.6 Scope Creep Risk

BIGGEST RISK

Adding V3 features (suspension telemetry, tire temp sensors, advanced ML) before V1 works reliably.

Mitigation: lock V1 deliverables by Week 2. Treat V2 and V3 as optional. Better to have a rock-solid core system than a feature-rich system that half-works.

09 R&D Zones — Candidate Solutions & Stretch Goals

9.1 Track Reconstruction Algorithm

TABLE 9.1 — CANDIDATE APPROACHES

CANDIDATE	METHOD	CON
A — Dead Reckoning	Integrate steering angle over time; add wheel speed for distance. Simple, no external dependencies	Accumulates error (~2–5 m per lap)
B — GPS + Kalman	Fuse GPS (±5–10 m) with wheel speed; Kalman filter reduces noise. Cost ≈ ₹1,500–2,000	Still ±1–2 m error
C — Optical Flow STRETCH	Small onboard camera, compute flow between frames; estimate velocity without GPS	Requires embedded computer vision (Raspberry Pi?), likely out of scope

9.2 Tire Grip Estimation from Telemetry

TABLE 9.2 — GRIP ESTIMATION OPTIONS

CANDIDATE	METHOD	CON
A — Cornering Stiffness Fitting	Assume a linear tire model (lateral force = Cf × slip angle); extract slip angle from steering + yaw rate; fit Cf to measured lateral acceleration. Result: quantified tire state per lap	Requires a high-fidelity IMU
B — Machine Learning V3	Train a neural net: [speed, steering, accel, yaw] → predicted grip. Needs 20–30 laps of diverse data	Overkill for V1; use the simpler physics model

9.3 Weather Compensation

TABLE 9.3 — COMPENSATION PATH

STAGE	APPROACH
Current	Manual input — pit operator notes “track is wet today”
V2 **Option**	BME280 temperature + humidity sensor onboard
V3 **Option**	Tire temperature sensing → infer grip from a thermal model

R&D question: can we reliably predict grip from ambient conditions alone? Probably not — track surface and rubber buildup matter too much.

9.4 Advanced Driver Modelling

TABLE 9.4 — DRIVER MODEL MATURITY

LEVEL	CAPABILITY
V1 **Simple**	Extract throttle/brake input from telemetry; use the raw pattern to seed the simulator
V2 **Advanced**	Classify driver behaviour per corner type — braking zone (aggressive vs. progressive), mid-corner (smooth vs. twitchy), acceleration zone (early vs. late throttle)
V3 **Stretch**	Predict driver behaviour from the start of a corner (anticipation model)

10 Achievable in Semester vs. Not Achievable

TABLE 10.1 — SCOPE LEDGER

TIER	DELIVERABLES
V1 **Weeks 1–6** ACHIEVABLE	ESP32 + 9 sensors working reliably • SD logging (full dataset, checksummed) • wireless live subset (10–20 Hz) to pit • driver display (steering-wheel OLED) • obstruction detection + TTC calculation • Python data pipeline (import, clean, sync) • point-mass simulator (track model + kart model) • calibration loop (adjust grip until model matches real lap ±5%) • pit dashboard (live gauges, basic plotting) • lap segmentation (manual or dead-reckoning based) • sector comparison (lap 1 vs. lap 2) • what-if simulation (speed sweep, surface change) • validation report (measured vs. predicted, error quantified)
V2 **Weeks 7–12** IF WELL-SCOPED	Brake pressure sensor integration • wireless pit dashboard (React frontend, real-time updates) • track reconstruction (GPS + Kalman filter) • session database (storage, query, historical comparison) • predictive ML model (simple neural net for grip) • video sync (telemetry overlay on lap video)
— **Out of reach** NOT ACHIEVABLE	Full vehicle dynamics simulator (suspension, load transfer, tire deflection) — use a commercial tool such as Project Chrono if needed • computer vision-based corner detection • RTK-GPS baseline (requires external hardware + survey) • autonomous control or braking assist • multi-camera 360° video system • production-grade enclosures (CNC, powder coat)

10.4 The Trade-Off Decision

TABLE 10.2 — DEPTH VS. BREADTH

OPTION	CONSEQUENCE
1 — Depth-First RECOMMENDED	Build V1 rock-solid. Dashboard, telemetry and simulator all trustworthy. Demo end-to-end: measure → model → validate → predict
2 — Breadth-First	Add V2 features quickly. More features to show judges, but less time for validation and testing

DECISION — **pick Option 1. A small, bulletproof system beats a large, flaky one.**

11 Semester Roadmap & Phase-Gated Timeline

11.1 Overall Cadence

Duration: 3–6 months (14–26 weeks), assuming 3–4 lab days per week per team member.
Principle: each phase must produce measurable, validated output before the next begins.

11.2 Phase-by-Phase Breakdown

TABLE 11.1 — PHASE GATES

PHASE	DELIVERABLES	TEAM MILESTONES	GO / NO-GO CRITERIA
1 — Foundation Weeks 1–2	ESP32 + sensor circuit on breadboard • individual sensor validation (wheel speed, RPM, temperature, IMU) • first SD card write (≥5 min reliable logging) • real-time serial plot	Embedded: all sensors reading correctly Data: CSV importer working Dashboard: pit receiver prototype ready	≥3 sensors logging without dropouts • timestamps aligned • no data corruption over a 10-minute test run
2 — Core Telemetry Weeks 3–4	Full 9-channel SD logging at 50–100 Hz • wireless live subset streaming to pit • driver OLED (speed, RPM, temp, TTC) • pit dashboard live gauges • first on-kart integration (wiring, mounts, enclosure)	Embedded: firmware feature-complete for V1 Integration: all sensors mounted on the actual kart Dashboard: live pit display working	Wireless data at pit with <200 ms latency • no SD write errors in a 15-min continuous test • driver display readable in sunlight • at least one successful track test
3 — Data Pipeline Weeks 5–6	Post-run import (SD → Python) • automated cleaning (outlier removal, gap filling) • lap segmentation • sector breakdown • comparison plots (lap 1 vs. lap 2 speed trace, G-force)	Python: pipeline tested on real track data Dynamics: started building the physics model	Import a 10 MB SD file in <5 s • identify ≥3 complete laps automatically • overlay two lap speed traces visually
4 — Digital Twin V1 Weeks 7–9	Point-mass kart model (mass, drag, power curve) • track model (centreline, curvature, elevation) • driver model extracted from telemetry • basic simulator predicting lap time • calibration routine	Dynamics: model running, producing speed traces Python: calibration loop working (error <5%)	Simulate a known corner within ±5% of measured • grip parameter converges on a real lap • explain the physical basis of the grip change between laps
5 — Validation & What-If Weeks 10–12	Model validated against 2–3 real laps • what-if scenarios (speed sweep, setup changes, surface conditions) • alternative line predicted and compared • documentation of assumptions, error sources, confidence limits	Dynamics: model trustworthy enough to publish results Testing: controlled experiments (e.g. “brake 50 m earlier”)	Model RMSE <5% across 3 test laps in different conditions • predictions consistent (same input → same output) • 3 what-if scenarios documented with measured validation
6 — Integration & Polish Weeks 13–14 (if needed)	Full end-to-end demo (run kart → collect → import → simulate → predict) • enhanced pit dashboard (session history, lap comparison) • technical report and user manual • video demo for judges	All teams: freeze, document, rehearse	System functions reliably from startup to final report generation

11.3 Milestone Calendar

TABLE 11.2 — WEEK-BY-WEEK OWNERSHIP

WEEK	EMBEDDED	DYNAMICS / PYTHON	DASHBOARD	INTEGRATION
1–2	Sensor bench test	Data import script	Pit receiver code	Thermal design

3–4	Firmware V1 complete	Raw data plots	Live dashboard	Enclosure 3D print
5–6	On-track test 1	Pipeline tested	Session replay	Wiring finalisation
7–9	Bug fixes, optimisation	Model calibration	Database schema	First wet run
10–12	Telemetry v1.1	Validation suite	Advanced features	Long-duration test
13–14	Release firmware	Final report	Polish UI	Demo prep

11.4 Weekly Team Sync

TABLE 11.3 — RECURRING CADENCE

WHEN	EVENT	AGENDA
Every Monday	30-minute all-hands standup	Each subteam: what got done, blockers, next week's plan • shared decision on scope/priority • track against the milestone calendar
Every Friday	Integration test	Run a quick kart session • check all layers work together (sensors → SD → pit → Python) • log bugs, plan fixes

12 Team Roles & Ownership

12.1 Functional Subteams

TABLE 12.1 — SEVEN SUBTEAMS

SUBTEAM	OWNER	RESPONSIBILITIES	KEY SKILLS
Embedded / Electronics	Name TBD	ESP32 firmware, sensor interfaces, signal conditioning, power management, wireless protocol	C/C++, embedded systems, circuit design, FreeRTOS
Mechanical / Integration	Name TBD	Sensor mounts, enclosures, wiring harness, vibration testing, thermal management	CAD (FreeCAD), 3D printing, mechanical design, EMI/RFI shielding
Telemetry / Networking	Name TBD	Pit receiver code, wireless packet format, live dashboard backend, session logging	Python, networking, real-time systems
Dashboard / UI	Name TBD	Frontend visualisation, live gauges, historical plotting, user experience	Python (Streamlit or React), UX design
Python / Data Analysis	Name TBD	Data import, cleaning, lap segmentation, sector analysis, FastF1 integration	Python (pandas, numpy, scipy), signal processing
Vehicle Dynamics / Simulation	Name TBD	Physics model, track geometry, kart parameters, calibration loop, what-if scenarios	Physics, numerical methods, MATLAB/Python, vehicle dynamics
Testing & Validation	Name TBD	Controlled track tests, measurement protocol, error quantification, benchmark documentation	Experimental design, data validation, technical writing

12.2 Cross-Functional Dependencies

TABLE 12.2 — INTERFACE FREEZE DATES

DEPENDENCY	DEADLINE	WHAT GETS FROZEN
Embedded ↔ Telemetry	Week 1	Wireless packet format agreed
Embedded ↔ Mechanical	Week 2	Enclosure & mount designs finalised
Python ↔ Dynamics	Week 3	Data format frozen
Dashboard ↔ Python	Week 4	API defined
Testing ↔ All	Every Friday	Track session logistics coordinated

12.3 Avoiding Bottlenecks

TABLE 12.3 — SINGLE POINTS OF FAILURE

RISK	MITIGATION
One person knows how to upload firmware	Cross-train immediately
Only one person has the GitHub repo password	Shared credentials
One person owns a critical design decision (e.g. track position algorithm)	Document assumptions
STANDING PRACTICE — **pair programming on critical modules • weekly documentation review • handoff exercises (A trains B, then A steps back)**	

12.4 Communication & Decision-Making

TABLE 12.4 — RHYTHM AND PROCESS

CADENCE	CHANNEL / ACTIVITY
Daily	Slack channel `#kart-engineering`

Weekly	Monday standup + Friday integration test
Monthly	Formal milestone review (if running longer than 16 weeks)

DECISION PROCESS

1. Problem identified by any subteam
2. Posted to `#kart-engineering` with context
3. Affected teams respond within 24 hours
4. Decision made by team leads at the Monday standup
5. Decision documented in the shared drive (rationale, trade-offs, date)

13 Cost Breakdown & Budget

13.1 V1 Complete BOM (Realistic)

TABLE 13.1 — V1 BILL OF MATERIALS

CATEGORY	ITEM	QTY	UNIT (₹)	SUBTOTAL (₹)
Controller	ESP32 Dev Board	1	500	500
Sensors	Hall wheel sensor + magnet	2	100	200
	Inductive RPM pickup	1	250	250
	DS18B20 temperature probe	1	100	100
	Rotary pot (steering)	1	75	75
	MPU6050 IMU	1	150	150
	HC-SR04 ultrasonic	1	100	100
	Throttle sensor (pot / Hall)	1	150	150
Data & Wireless	MicroSD module	1	150	150
	MicroSD card (32 GB)	1	300	300
	nRF24L01+ module	1	200	200
	External antenna	1	80	80
Display & Alerts	0.96" OLED (I2C)	1	200	200
	Active buzzer	1	30	30
Power & Protection	7805 regulator + caps	1	250	250
	Battery isolation capacitors	1	100	100
Wiring & Mounting	Shielded twisted pair	50 m	20	1,000
	Connectors (M5 plugs, etc.)	—	—	400
	Perfboard, breadboard	—	—	300
	3D printed mounts	—	—	500
Enclosures & Fasteners	Aluminium case (sensor / controller)	1	600	600
	Mounting hardware	—	—	200
	Vibration dampeners	—	—	300
Software & Tools	Git + GitHub	—	0	0
	Arduino IDE / ESP-IDF	—	0	0
	Python libraries	—	0	0
V1 TOTAL				**7,935–9,500**

WHAT THIS TOTAL EXCLUDES

- Second-tier components (brake pressure sensor, GPS, advanced IMU)
- Specialised test equipment (oscilloscope, signal analyser)
- Recurring costs (fuel, track rental, travel)

13.2 V2 & V3 Expansion Costs

TABLE 13.2 — EXPANSION BUDGET

ITEM	COST (₹)	TIER
Brake pressure sensor	800	V2
GPS / GNSS module	1,500	V2
Secondary wheel sensors	400	V2
ICM-42688 IMU (premium)	600	V2
VL53L0X ToF sensor	400	V2
Development & integration	5,000	V2
V2 SUBTOTAL	**≈ 8,000–12,000**	
Suspension travel sensors	2,000	V3
Tire temperature sensors	3,000	V3
Tire pressure sensors	1,500	V3
RTK-GPS module (rental or purchase)	8,000–15,000	V3
Advanced ML / Chrono simulator	5,000	V3
V3 SUBTOTAL	**15,000+**	

13.3 Budget Optimisation

TO STAY WITHIN ₹15,000–20,000 TOTAL

- Buy sensors piecemeal — don't bulk-order everything upfront
- Use free software (Arduino IDE, Python)
- 3D print enclosures rather than CNC machining them
- Share expensive equipment (oscilloscope) among subteams
- Defer V3 features until after V1 validation

14 Realistic vs. Aspirational Outcomes

TABLE 14.1 — DELIVERY CONFIDENCE LEDGER

CONFIDENCE	ITEM	DETAIL
WILL DELIVER	Telemetry	Full 9-channel logging at 50–100 Hz, on-board storage, live wireless subset to pit
	Driver assistance	Steering-wheel display with speed, RPM, temperature, TTC warning
	Data pipeline	Import, clean, segment and analyse real track data
	Physics model	Calibrated simulator predicting corner speeds within ±5%
	Validation	Measured error quantified; simulator vs. reality comparison documented
	Integration	End-to-end system: run kart → get data → simulate → predict
MIGHT DELIVER	Advanced track position	Sensor fusion reducing GPS error to ±1–2 m
	Predictive what-if	Fast scenario testing (speed sweep, setup change, wet surface)
	Session database	Historical lap comparison, trend analysis
	Video sync	Telemetry overlay on track video footage
	Machine learning	Neural net predicting tire grip from telemetry
WILL NOT DELIVER	Full vehicle dynamics	Suspension load transfer, tire slip circle, multi-body dynamics
	Autonomous braking	The system advises; the driver controls
	Production enclosures	3D prints and aluminium cases, not powder-coated housings
	Real-time GNSS	We accept ±1–5 m error; no RTK-GPS integration in V1
	Computer vision	No camera-based corner detection or vision odometry

14.4 The Honest Pitch to Judges

> THE PITCH
>
> "We don't just collect telemetry — we convert it into a validated engineering model that predicts kart behaviour and guides setup optimisation. The closed loop is the innovation: measure → model → predict → validate → improve."

TABLE 14.2 — HONESTY POINTS

WE DO	INSTEAD OF
Quantify model error	Claiming "it's perfect"
Measure latency	Claiming "it's instant"
Identify open problems (track position, tire grip estimation)	Hiding the unsolved parts
Scope realistically — V1 small, V2 ambitious, V3 dreams	Promising everything at once

15 Technical Documentation & Knowledge Base

15.1 Living Document Standards

TABLE 15.1 — FILES EVERY SUBTEAM MAINTAINS

FILE	CONTENTS
`DESIGN.md`	Architecture decisions, rationale, trade-offs
`API.md`	Data formats, interfaces, communication protocols
`BUILD.md`	Hardware assembly, pin mappings, calibration procedures
`TEST.md`	Validation procedures, expected results, pass/fail criteria

15.2 GitHub Organisation

```
kurukshetra-kart/
├── README.md                    (overview, getting started)
├── docs/
│   ├── SYSTEM_ARCHITECTURE.md
│   ├── HARDWARE_GUIDE.md
│   ├── SOFTWARE_API.md
│   ├── PHYSICS_MODEL.md
│   └── VALIDATION_REPORT.md
├── firmware/
│   ├── src/
│   │   ├── main.cpp
│   │   ├── sensors.cpp
│   │   ├── telemetry.cpp
│   │   └── display.cpp
│   └── calibration_data.h
├── python/
│   ├── kurukshetra_kart/
│   │   ├── __init__.py
│   │   ├── session.py
│   │   ├── simulator.py
│   │   └── analysis.py
│   ├── tests/
│   │   └── test_pipeline.py
│   └── notebooks/
│       └── example_lap_analysis.ipynb
├── dashboard/
│   └── pit_app.py               (Streamlit)
└── hardware/
    ├── schematics/
    ├── pcb_layout/
    └── 3d_models/
```

15.3 Code Review & Quality Standards

TABLE 15.2 — STANDARDS BY DOMAIN

DOMAIN	REQUIREMENT
Python	PEP 8 style, docstrings on every function, unit tests for the data pipeline
C++	Comments on every non-obvious calculation, watchdog timer in firmware, graceful error handling
Documentation	Timestamp every design decision; explain “why”, not just “what”

16 Final Recommendations & Success Criteria

16.1 Success = Closed Loop, Not Feature Count

TABLE 16.1 — WHAT JUDGES WEIGH

WEIGHT	CRITERION
CARE MOST	The system actually works — telemetry collected, stored, retrieved without errors
	The model is validated — predicted behaviour matches measured behaviour, quantified
	The loop is closed — real kart data feeds the simulator, the simulator predicts, the prediction is tested on the kart
	Honesty — errors acknowledged, limitations stated, methodology transparent
CARE LEAST	Fancy UI (basic Streamlit is fine)
	Perfect hardware packaging (3D prints are okay)
	Every possible sensor (9 core channels is enough)
	Unrealistic performance claims ("predicts lap time to the millisecond")

16.2 Semester Survival Strategy

TABLE 16.2 — MONTH-BY-MONTH FOCUS

PERIOD	THEME	OBJECTIVES
Month 1 — Weeks 1–4	Foundation	Get every sensor working • first on-kart test • establish the data pipeline
Month 2 — Weeks 5–8	Core system	Wireless telemetry reliable • physics model calibrated • pit dashboard live
Month 3 — Weeks 9–12	Validation & refinement	Model error <5% • what-if scenarios tested • documentation complete

IF THINGS SLIP — **cut from V2/V3 and polish V1 instead. A tight, validated core beats a rushed, feature-rich mess.**

16.3 Demonstration Sequence (For Judges)

TABLE 16.3 — DEMO RUNNING ORDER

#	SEGMENT	WHAT HAPPENS
1	**Live track run**	Kart runs; live telemetry arrives at the pit dashboard
2	**Data walkthrough**	Import SD card → show raw data, cleaned data, lap segmentation
3	**Model calibration**	"Here's lap 1. We calibrate grip to it. Now here's lap 2 — the model predicts corner speeds. Here's the error."
4	**What-if scenario**	"If we did this corner 5 km/h faster, the simulator predicts this much more grip is needed. Here's the measurement from a real try."
5	**Honest limitations**	"Track position is ±2 m error. Tire grip estimation is ±10%. These are engineering truths we accept."

ONE-LINE PITCH

"Real kart. Real data. Real model. Real validation. The closed loop."

A Appendix A — ESP32 Pin Mapping

TABLE A.1 — ANALOG & DIGITAL ASSIGNMENTS

PIN	TYPE	ASSIGNMENT
GPIO 35	ADC	Wheel speed sensor (Hall voltage)
GPIO 34	ADC	Throttle position
GPIO 32	ADC	Steering angle potentiometer
GPIO 33	ADC	Engine temperature (via OpAmp from DS18B20)
GPIO 26	PWM	OLED backlight (optional dimming)
GPIO 25	Digital	Buzzer / alert tone
GPIO 27	Digital	Status LED
GPIO 14	Interrupt	RPM inductive pickup (pull-up, debounce in firmware)

TABLE A.2 — BUS ASSIGNMENTS

BUS	PINS	DEVICES
I2C	SDA GPIO 21 / SCL GPIO 22	OLED, MPU6050, DS18B20 adapter
SPI — SD card	MISO 19, MOSI 23, CLK 18, CS 5	MicroSD logging module
SPI — wireless	CE 17, CSN 16	nRF24L01+ (shares the SPI bus, separate CS)
UART — debug	TX 1 / RX 3	USB serial adapter

CRITICAL PIN NOTES

- Use GPIO 36 & 39 for input only — they lack pull-up/down
- Reserve GPIO 4 & 0 as system pins — don't use for sensors
- Avoid GPIO 6–11 — reserved for the flash memory interface
- The SPI bus can multiplex SD card + wireless with separate CS pins

B Appendix B — FastF1 Throttle Pattern Extraction

```
import fastf1
import pandas as pd
import matplotlib.pyplot as plt

# Fetch F1 session (example: 2023 Monaco, Verstappen qualifying lap)
fastf1.Cache.enable_cache('/tmp/f1_cache')
session = fastf1.get_session(2023, 'Monaco', 'Q')
session.load()

# Get telemetry for one driver
driver    = 'VER'
lap       = session.laps.pick_driver(driver)[0]
telemetry = lap.get_car_data()

# Extract throttle pattern
throttle_pattern = telemetry[['Distance', 'Speed', 'Throttle', 'Brake']].copy()

# Plot to inspect
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))

ax1.plot(throttle_pattern['Distance'], throttle_pattern['Speed'])
ax1.set_ylabel('Speed (km/h)')
ax1.set_title(f'{driver} - Speed Trace')
ax1.grid(True)

ax2.plot(throttle_pattern['Distance'], throttle_pattern['Throttle'], label='Throttle')
ax2.plot(throttle_pattern['Distance'], throttle_pattern['Brake'],    label='Brake')
ax2.set_xlabel('Distance (m)')
ax2.set_ylabel('Input (%)')
ax2.legend()
ax2.set_title(f'{driver} - Driver Input')
ax2.grid(True)

plt.tight_layout()
plt.savefig('f1_lap_pattern.png', dpi=150)
plt.show()

# Seed the kart simulator:
#   "At D=500 m, the F1 car does 150 km/h with 80% throttle"
#   "At D=500 m (similar corner), what speed do WE do with 80% throttle?"
#   Compare -> calibrate kart model
```

C Appendix C — Validation Checklist

> Use this checklist weekly during the build and before every track test.

TABLE C.1 — HARDWARE VALIDATION

☐	CHECK
☐	All 9 sensors reading correctly (serial monitor)
☐	Timestamps aligned — all channels stamped at the same microsecond
☐	SD write stable for 30 minutes (no corruption, checksum passes)
☐	Wireless range ≥100 m line-of-sight
☐	OLED display readable in sunlight
☐	Buzzer audible at track noise level

TABLE C.2 — FIRMWARE VALIDATION

☐	CHECK
☐	Compiles without errors (Arduino IDE or ESP-IDF)
☐	Uploads to ESP32 without issues
☐	Watchdog timer active; resets the system if firmware hangs
☐	No memory leaks — heap size stable after 1 hour runtime
☐	Graceful degradation — if wireless fails, the kart keeps logging

TABLE C.3 — DATA PIPELINE VALIDATION

☐	CHECK
☐	Import SD file in <5 seconds
☐	Detect and skip corrupted records
☐	Interpolate small data gaps (<500 ms)
☐	Timestamp continuity check — no jumps backward
☐	Lap detection works (automated or manual)

TABLE C.4 — MODEL VALIDATION

☐	CHECK
☐	Grip calibration converges — error decreases over iterations
☐	Predicted speed ≥ measured speed in corners (conservative)
☐	Model RMSE <5% on the test lap
☐	What-if scenario produces sensible results (speed change → time change in the right direction)

TABLE C.5 — INTEGRATION VALIDATION

☐	CHECK
☐	Run the kart for 5 laps; collect the full dataset
☐	Import, clean and analyse data without crashes

☐	Dashboard displays live telemetry correctly
☐	Pit and kart clocks synced — time difference <100 ms
☐	End-to-end demo works: run → collect → import → plot → simulate

D Appendix D — Track Position Dead Reckoning

> For V1 (Weeks 5–6), implement simple dead reckoning to estimate lap position.

```
import numpy as np

class DeadReckoner:
    def __init__(self, wheelbase=1.2, track_length=1000.0):
        """
        wheelbase    : distance from rear axle to front axle (meters)
        track_length : approximate perimeter (meters)
        """
        self.wheelbase    = wheelbase
        self.track_length = track_length

        self.x            = 0.0    # East position
        self.y            = 0.0    # North position
        self.theta        = 0.0    # Heading (radians)
        self.lap_distance = 0.0
        self.lap_count    = 0

    def update(self, wheel_speed_ms, steering_angle_rad, dt):
        """
        wheel_speed_ms     : speed from wheel sensor (m/s)
        steering_angle_rad : steering angle (radians)
        dt                 : time step (seconds)

        Returns: (lap_distance, lap_count)
        """
        v     = wheel_speed_ms
        delta = steering_angle_rad

        # Kinematic bicycle model
        #   dx/dt     = v * cos(theta)
        #   dy/dt     = v * sin(theta)
        #   dtheta/dt = (v / wheelbase) * tan(delta)
        self.theta += (v / self.wheelbase) * np.tan(delta) * dt
        self.x     += v * np.cos(self.theta) * dt
        self.y     += v * np.sin(self.theta) * dt

        # Distance travelled
        self.lap_distance += v * dt

        # Detect lap boundary
        if self.lap_distance > self.track_length:
            self.lap_distance -= self.track_length
            self.lap_count    += 1

        return self.lap_distance, self.lap_count


# Usage in the main telemetry loop
dr = DeadReckoner(wheelbase=1.2, track_length=1000.0)

for record in telemetry_log:
    speed    = record['wheel_speed_kmh'] / 3.6            # km/h -> m/s
    steering = record['steering_angle_deg'] * np.pi / 180 # deg  -> rad
    dt       = 0.01                                       # 100 Hz sampling

    lap_dist, lap_num = dr.update(speed, steering, dt)
    print(f"Lap {lap_num}, Distance {lap_dist:.1f} m")
```

TABLE D.1 — LIMITATIONS & IMPROVEMENT PATH

ASPECT	DETAIL
Drift	Error accumulates over time (~±2–5 m per lap)
Input quality	Steering angle must be accurate
Assumption	No tire slip — any skid invalidates the model
Improvement V2	Combine with GPS using a Kalman filter

Closing Statement

> This document is the team's blueprint for building a complete, validated telemetry-to-prediction system in one semester. It prioritises **integration and validation over feature count**. Success is a working closed loop: measure → model → predict → test → improve.

TABLE 17.1 — KEY COMMITMENTS

COMMITMENT	STATEMENT
Realistic cost	₹18,000–24,000 for V1
Single controller	ESP32 only — no Arduino Nano
Honest error reporting	Model RMSE quantified, not hidden
Semester-scale timeline	3–6 months, phase-gated
Clear team ownership	No overlapping authority
Flagged R&D zones	Track position and tire grip, each with candidate solutions
Layered delivery	Achievable V1, ambitious V2, dream V3

Go build something great.

Kurukshetra Racing, SKCET • Revision 1.0 — Master Build Manual • September 2026